LINFO 1121
DATA STRUCTURES AND ALGORITHMS

UCLouvain

Restructuration 1
Linear Data-Structures and Time Complexity

Pierre Schaus

1.2.1

- Iterable vs Iterator

- Implementation of a CircularLinkedList

1.2.1: Iterable vs Iterator (True or False)

- Iterator is an interface that represents a collection of elements that can be iterated over. It provides the method iterator() that returns an Iterator object (true or false ?)

1.2.1: Iterable vs Iterator (True or False)

- ~~Iterator~~ Iterable is an interface that represents a collection of elements that can be iterated over. It provides the method iterator() that returns an Iterator object

1.2.1: Iterable vs Iterator (True or False)

- Iterator is an interface that provides methods (hasNext(), next(), and remove()) to traverse or iterate over the elements in a collection (true or false ?)

1.2.1: Iterable vs Iterator (True or False)

- Iterator is an interface that provides methods (hasNext(), next(), and remove()) to traverse or iterate over the elements in a collection. ✅

1.2.1: Iterable vs Iterator (True or False)

- The code on the left is a syntactical sugar for the code on the right (true or false ?)

```
for (Integer i: collection) {
    System.out.println(i);
}
```

```
Iterator<Integer> iterator = collection.iterator();
while (iterator.hasNext()) {
    System.out.println(iterator.next());
}
```

- Let’s verify it by looking at the decompiled code in IntelliJ …

1.2.1: Iterable vs Iterator (True or False)

- This is the proper way of implementing an iterator for my collection ?

```
import java.util.Iterator;

class MyCollection implements Iterable<Integer>, Iterator<Integer> {
    private int[] data = {1, 2, 3, 4, 5};

    private int index = 0;


    @Override
    public Iterator<Integer> iterator() {
        return this;
    }

    @Override
    public boolean hasNext() {
        return index < data.length;
    }

    @Override
    public Integer next() {
        return data[index++];
    }
}
```

1.2.1: Iterable vs Iterator (True or False)

- No ☠️❌☠️❌☠️

```
import java.util.Iterator;

class MyCollection implements Iterable<Integer>, Iterator<Integer> {
    private int[] data = {1, 2, 3, 4, 5};

    private int index = 0;


    @Override
    public Iterator<Integer> iterator() {
        return this;
    }

    @Override
    public boolean hasNext() {
        return index < data.length;
    }

    @Override
    public Integer next() {
        return data[index++];
    }
}
```

What do you think this will print ?

```
public static void main(String[] args) {
    MyCollection collection = new MyCollection();
    for (int i : collection) {
        for (int j : collection) {
            System.out.println(i + "," + j);
        }
    }
}
```

1,2
1,3
1,4
1,5

1.2.1: Iterable vs Iterator (True or False)

- This is the proper way 👇👌✅

```java
import java.util.Iterator;

class MyCollection implements Iterable<Integer> {
    private int[] data = {1, 2, 3, 4, 5};

    @Override
    public Iterator<Integer> iterator() {
        return new MyIterator();
    }

    // Separate Iterator class
    private class MyIterator implements Iterator<Integer> {
        private int index = 0;

        @Override
        public boolean hasNext() {
            return index < data.length;
        }

        @Override
        public Integer next() {
            return data[index++];
        }
    }
}
```

With an inner-class that contains its own state (index instance variable)

```java
public static void main(String[] args) {
    MyCollection collection = new MyCollection();
    for (int i : collection) {
        for (int j : collection) {
            System.out.println(i + "," + j);
        }
    }
}
```

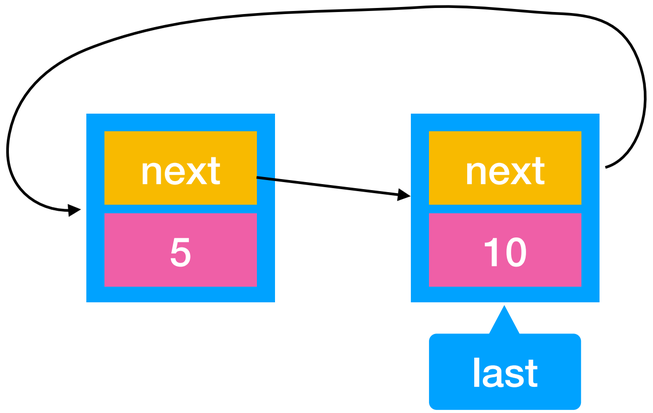
1,1
1,2
1,3
1,4
1,5
2,1
2,2
2,3
2,4
...
5,5

Les private inner classes sont très utiles pour implémenter des iterateurs car celles-ci ont accès à l'état de l'outer class

1.2.1 CircularLinkedList, ChatGPT Solution 🤔

```java
public class CircularLinkedList<Item> implements Iterable<Item> {

    private int n;          // size of the list
    private Node last;      // trailer of the list (points to the last node)

    private class Node {
        private Item item;
        private Node next;
    }
    public CircularLinkedList() {
        n = 0;
        last = null;
    }
    public boolean isEmpty() {
        return n == 0;
    }
    public int size() {
        return n;
    }
    public void enqueue(Item item) {
        Node newNode = new Node();
        newNode.item = item;
        if (isEmpty()) {
            newNode.next = newNode;   // Points to itself since it's the only node
            last = newNode;
        } else {
            newNode.next = last.next; // The new node points to the first node
            last.next = newNode;      // The current last node points to the new node
            last = newNode;           // Update the last pointer to the new node
        }
        n++;  // Increase the size of the list
    }
}
```

Two elements

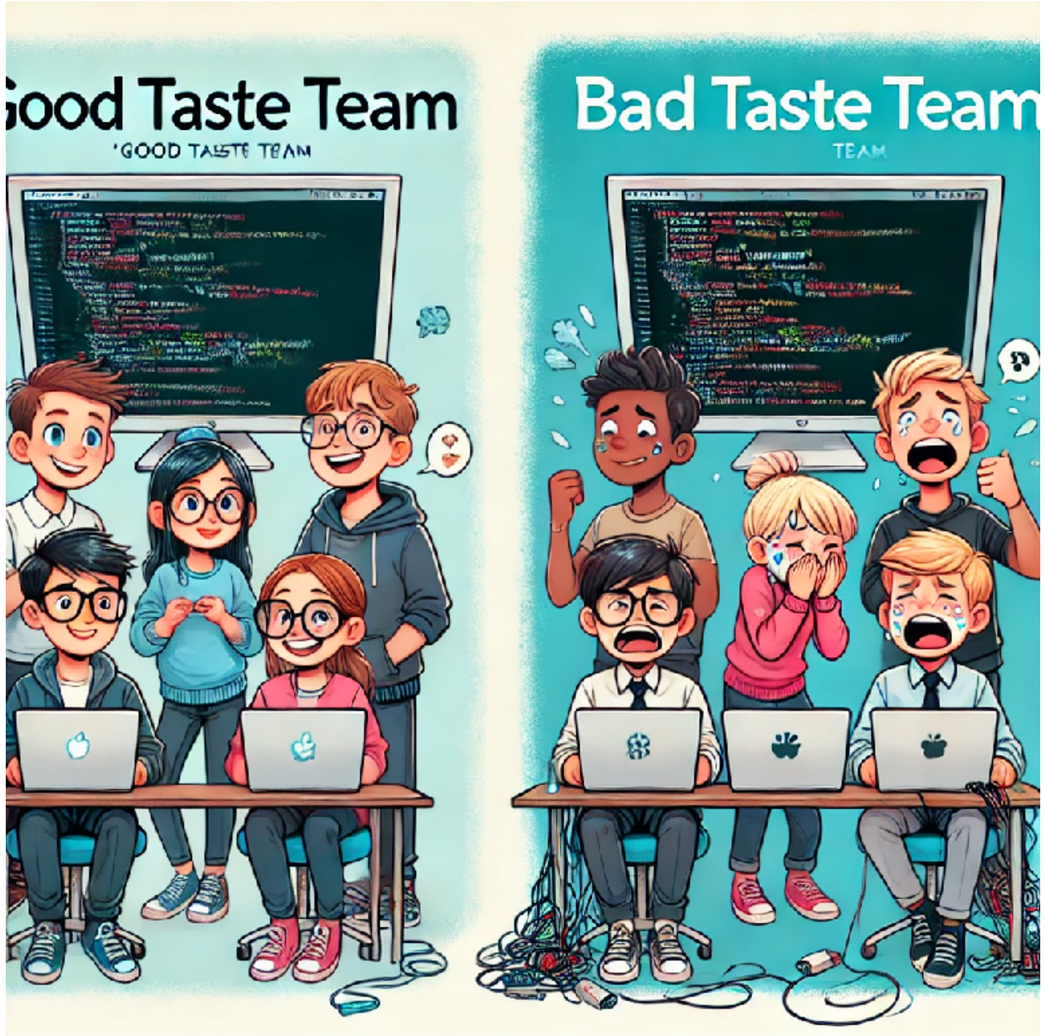



```java
public Item remove(int index) {
    if (index < 0 || index >= n) {
        throw new IndexOutOfBoundsException();
    }

    Node current = last.next;  // Start from the first node
    Item removedItem;

    if (index == 0) {  // Removing the first node
        removedItem = current.item;
        if (n == 1) {  // If there's only one node
            last = null; // The list becomes empty
        } else {
            last.next = current.next; // Bypass the first node
        }
    } else {
        Node prev = last.next;  // Start from the first node
        for (int i = 0; i < index - 1; i++) {
            prev = prev.next;
        }
        current = prev.next;  // The node to remove
        removedItem = current.item;
        prev.next = current.next; // Bypass the node to remove
        if (current == last) {    // If the last node is removed
            last = prev;
        }
    }
    n--;   // Decrease the size
    return removedItem;
}
```

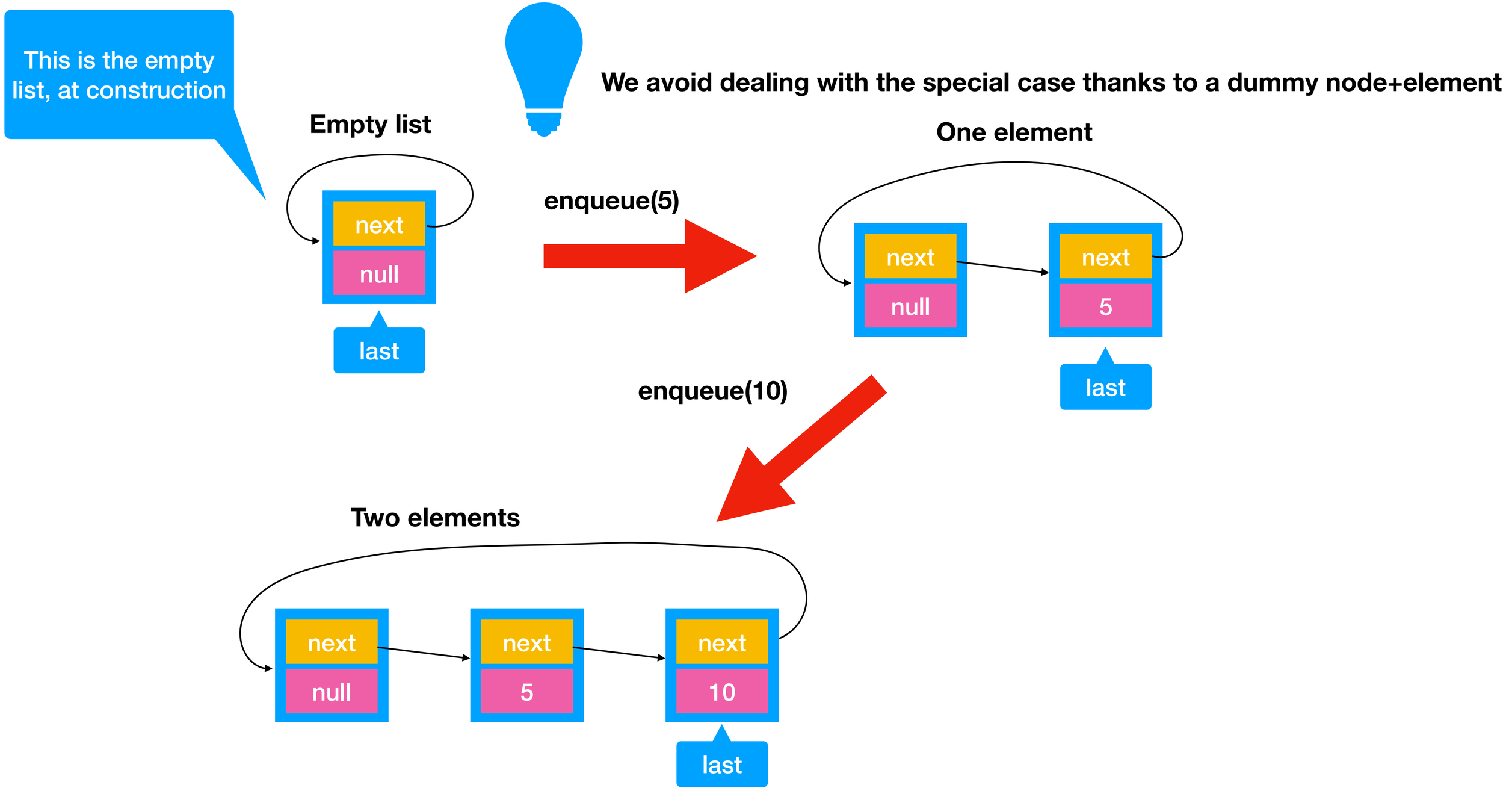
Good Taste in Coding (Linus Torvalds, creator of Linux)

What is your team ?

- Then think a bit ahead before diving in the code, use a 📝 and draw!



1.2.1 CircularLinkedList, let's get rid of the special cases



1.2.1 CircularLinkedList, let's get rid of the special cases

Now (Good Taste Solution)

```
public class CircularLinkedList<Item> implements Iterable<Item> {

   private int n;           // size of the stack
    private Node  last;     // trailer of the list

    // helper linked list class
    private class Node {
        private Item item;
        private Node next;
    }

    public CircularLinkedList() {
        last = new Node();
        last.next = last;
        n = 1;
    }

    public boolean isEmpty() {
        return n == 1;
    }
    public int size() {
        return n-1;
    }
    private long nOp() {
        return nOp;
    }

    public void enqueue(Item item) {
        Node oldLast = last;
        last = new Node();
        last.item = item;
        last.next = oldLast.next;
        oldLast.next = last;
        n++;

    }
}
```

Before

```
public void enqueue(Item item) {
    Node newNode = new Node();
    newNode.item = item;
    if (isEmpty()) {
        newNode.next = newNode;
        last = newNode;
    } else {
        newNode.next = last.next;
        last.next = newNode;
        last = newNode;
    }
    n++;
}
```

1.2.1 CircularLinkedList, let's get rid of the special cases

Good Taste

```
public Item remove(int index) {
    if (index < 0 || index >= size()) {
        throw new IndexOutOfBoundsException();
    }
    Node prev = last.next;
    for (int i = 0; i < index; i++) {
        prev = prev.next;
    }
    if (prev.next == last) {
        last = prev;
    }
    Item v = prev.next.item;
    prev.next = prev.next.next;
    n--;
    return v;
}
```

Before (Bad Taste)

```
public Item remove(int index) {
    if (index < 0 || index >= n) {
        throw new IndexOutOfBoundsException();
    }

    Node current = last.next;  // Start from the first node
    Item removedItem;

    if (index == 0) {  // Removing the first node
        removedItem = current.item;
        if (n == 1) {  // If there's only one node
            last = null; // The list becomes empty
        } else {
            last.next = current.next; // Bypass the first node
        }
    } else {
        Node prev = last.next;  // Start from the first node
        for (int i = 0; i < index - 1; i++) {
            prev = prev.next;
        }
        current = prev.next;  // The node to remove
        removedItem = current.item;
        prev.next = current.next; // Bypass the node to remove
        if (current == last) {    // If the last node is removed
            last = prev;
        }
    }
    n--;   // Decrease the size
    return removedItem;
}
```

1.2.1 CircularLinkedList

- What is the time complexity of de: `public void enqueue(Item item)` ?
  - O(1)?
  - O(n) where n = number of of entries in the list?
  - $\Theta(n)$ where n = number of of entries in the list?
  - O(1) amortized?

1.2.1 CircularLinkedList

- Complexité de: `public Item remove(int index)` ?
  - O(1)
  - O(n) où n est le nombre d'entrées dans la liste
  - $\Theta(n)$ où n est le nombre d'entrées dans la liste
  - O(1) ammorti

1.2.1 Reminder: Concurrent Modification

- A collection generally cannot be modified while an iterator is being used on it. This is the *Fail-Fast* strategy.

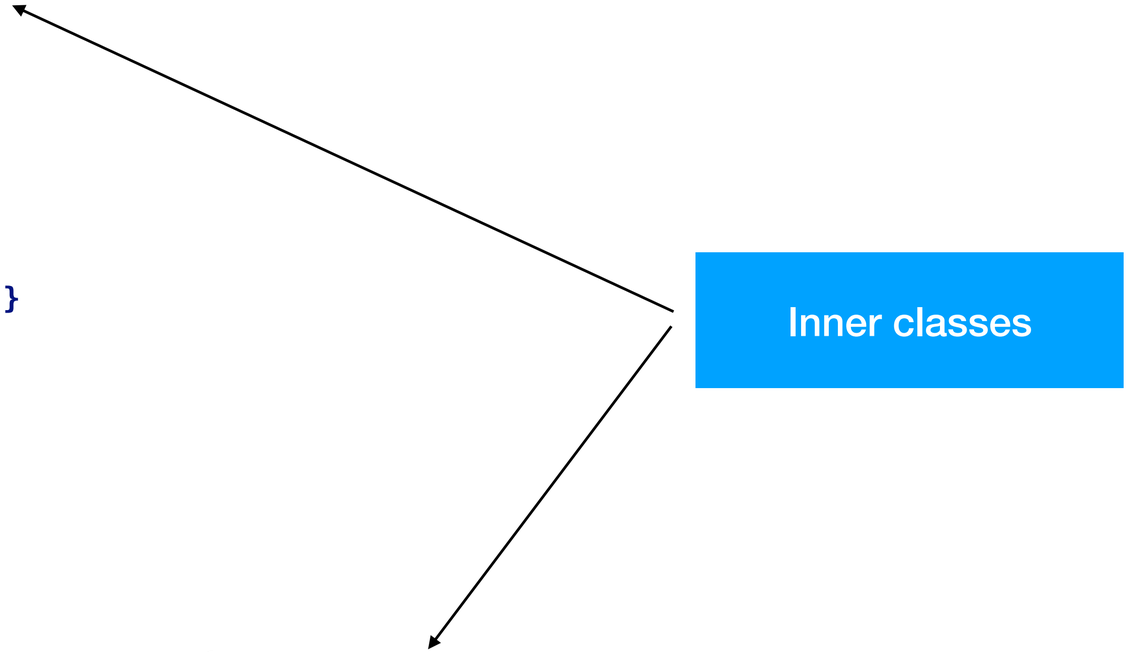
- If the collection is modified between calls to `hasNext`/`next`, a `ConcurrentModificationException` must be thrown.

```
List<Integer> collection = new LinkedList<>();
collection.add(1);
collection.add(2);
for (Integer i: collection) {
    collection.add(i);
}
```

Exception in thread "main"
java.util.ConcurrentModificationException

1.2.1 CircularLinkedList

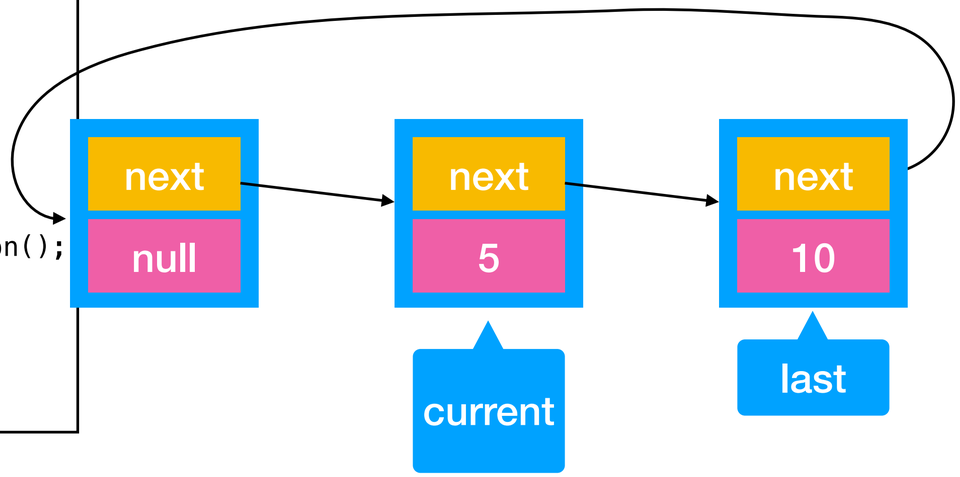
```
public class CircularLinkedList<Item> implements Iterable<Item> {

    private int n;           // size of the stack
    private Node  last;      // trailer of the list

    private class Node {
        private Item item;
        private Node next;
    }

    public CircularLinkedList() {
        last = new Node();
        last.next = last;
        n = 1;
    }
    public boolean isEmpty() { return n == 1; }
    public int size() {
        return n-1;
    }
    public void enqueue(Item item) { … }
    public Item remove(int index) { … }

    public Iterator<Item> iterator() {
        return new ListIterator();
    }

    private class ListIterator implements Iterator<Item> {
        private ListIterator() { … }
        public boolean hasNext()  { … }
        public void remove() {
            throw new UnsupportedOperationException();
        }
        public Item next() { … }
    }
}
```



1.2.1

```
public class CircularLinkedList<Item> implements Iterable<Item> {

    private long nOp = 0; // count the number of operations
    private int n;          // size of the stack
    private Node  last;   // trailer of the list

    public Iterator<Item> iterator() {
        return new ListIterator();
    }

    private class ListIterator implements Iterator<Item> {

        private Node current;

        private ListIterator() {
           current = last.next.next;
        }

        public boolean hasNext()  {
           return current != last.next;
        }

        public void remove()      {
            throw new UnsupportedOperationException();
        }

        public Item next() {
            if (!hasNext()) throw new NoSuchElementException();
            Item item = current.item;
            current = current.next;
            return item;
        }
    }

}
```

L'itérateur a donc son propre état interne (variable d'instance current propre à l'instance de itérateur au sein d'une instance particulière d'une CircularLinkedList) qui est le noeuds avec le prochain item à retourner



Et le ConcurrentModificationException ?

- On va stocker un compteur interne à la liste qui compte les operations « enqueue » et « remove »

- A chaque operation « enqueue » ou « remove » on va augmenter ce compteur.

- Au moment de créer l'itérateur, on va y stocker (variable d'instance) la variable du compteur.

- Si lorsqu'on fait « hasNext() » ou « next() », on réalise que la valeur du « compteur » enregistrée dans l'itérateur est plus petite que celle du compteur de la liste, cela signifie que l'utilisateur a modifié la liste alors qu'il est en train d'utiliser l'itérateur => ConcurrentModificationException

Iterator with detection of modification on the data-structure

```
public class CircularLinkedList<Item> implements Iterable<Item> {

    private long nOp = 0;    // count the number of operations
    private int n;           // size of the stack
    private Node  last;      // trailer of the list

    …

    private long nOp() {
        return nOp;
    }

    public void enqueue(Item item) {
        nOp++;
        …
    }
    public Iterator<Item> iterator() {
        return new ListIterator();
    }
```

Increment number of operations

```
private class ListIterator implements Iterator<Item> {

    private Node current;
    private long nOp;

    private ListIterator() {
        nOp = nOp();
        current = last.next.next;
    }

    public boolean hasNext()  {
        if (nOp() != nOp) throw new ConcurrentModificationException();
        return current != last.next;
    }
}
```

Snapshot of the number of operations on the list at the time the iterator is created

Number of operations on the list at the time of iterator creation

Number of operations on the list at the present time

```
}
```

CircularLinkedList

- What is the time complexity of creating an iterator and then iterating over the first k elements?
  - O(n) where *n* is the number of entries in the list
  - Θ(n) where *n* is the number of entries in the list
  - O(k)
  - Θ(k)

1.2.2 Implementation of a Stack with an

- Array

- LinkedList

1.2.2 Implementation of a Stack with an Array

Strategy: Doubling the size of the array when reaching it's capacity. How ?

```
class ArrayStack<E> implements Stack<E> {

    private E[] array;          // array storing the elements on the stack
    private int size;          // size of the stack

    public ArrayStack() {
        array = (E[]) new Object[10];
    }
    @Override
    public boolean empty() {
        return size == 0;
    }
    @Override
    public E peek() throws EmptyStackException {
        if (empty()) throw new EmptyStackException();
        else return array[size-1];
    }
    @Override
    public void push(E item) {
        if (size == array.length) {
            resize(array.length * 2); // doubling the size of the array
        }
        array[size++] = item;
    }
    // pop on next slide
    private void resize(int newCapacity) {
        E[] newArray = (E[]) new Object[newCapacity];
        System.arraycopy(array, 0, newArray, 0, size);
        array = newArray;
    }
}
```

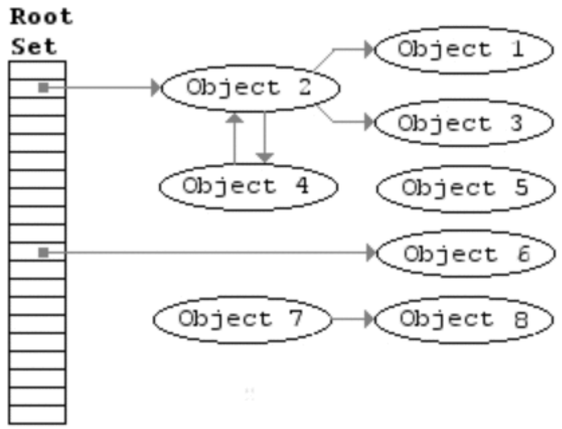
Might be more efficient than a for loop for copying, but still Θ(n)

1.2.2 Implementation of a Stack with an Array

Why resizing for the pop ?

```
class ArrayStack<E> implements Stack<E> {

    @Override
    public E pop() throws EmptyStackException {
        if (empty()) throw new EmptyStackException();
        else {
            E ret = array[size-1];
            size--;
            array[size] = null; // so that the object can possibly be garbage collected
            // Resize the array if size is less than half the length
            if (size > 0 && size <= array.length / 2) {
                resize(array.length / 2);
            }
            return ret;
        }
    }
}
```

To allow the garbage collector to reclaim unused memory as soon as possible



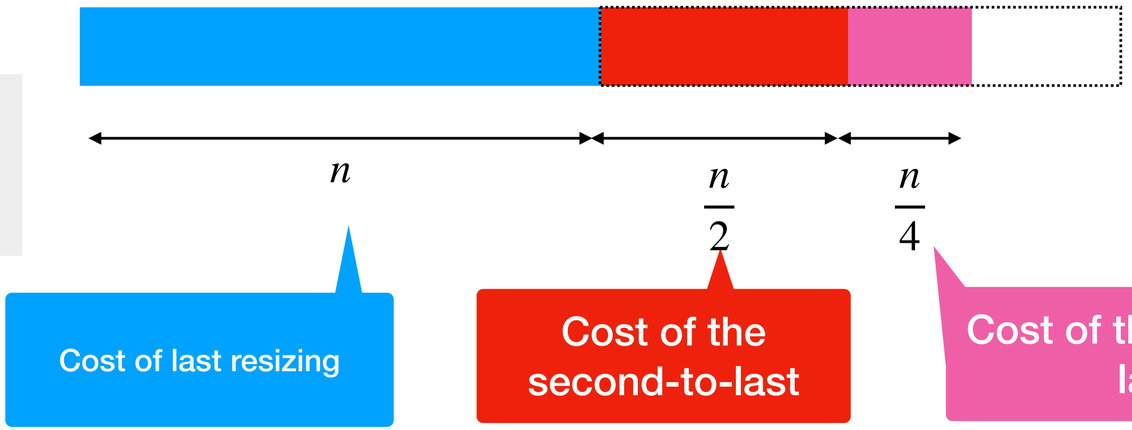
1.2.2 Stack with an array: Time Complexity Reminder

	Liste chainée	Tableau
Push	O(1)	O(1) si pas de redim O(n) si redim O(1) amortized
Pop	O(1)	O(1) si pas de redim O(n) si redim O(1) amortized
n push	O(n)	O(n)
n pop	O(n)	O(n)

The array size is doubled when its capacity limit is reached.

Thus, for *n* push operations, resizing occurs at these positions:

- 1
- 2
- 4
- 8
- 16
- ...
- n

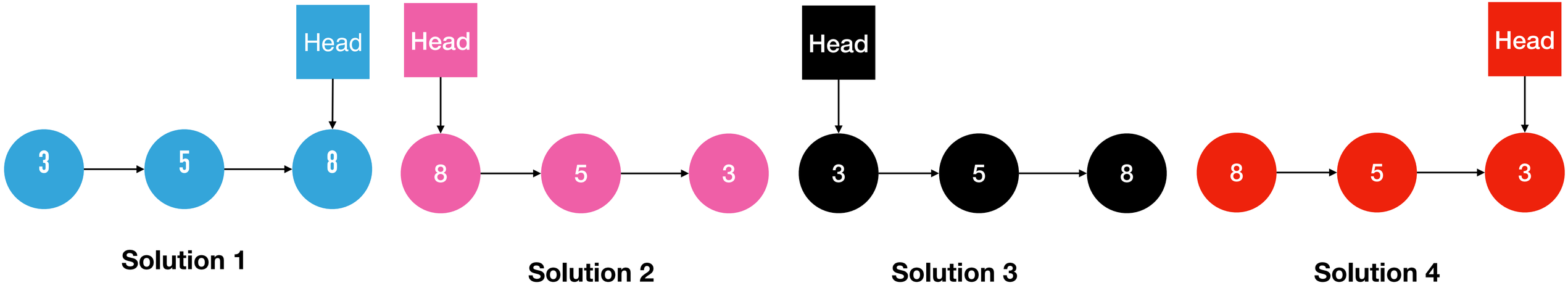


1.2.2 Stack with a simply LinkedList

Operations:

push(3)
push(5)
push(8)

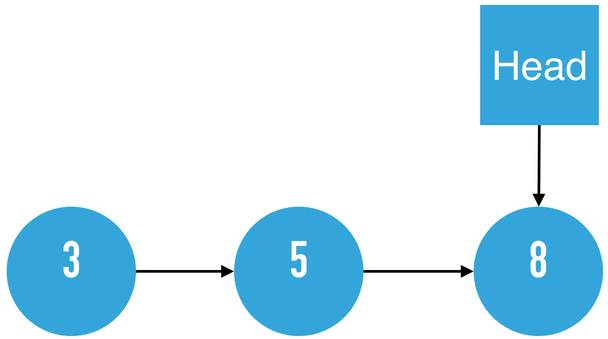
Where should we place the head such that the push/pop can be performed efficiently?



1.2.2 Stack with a simply LinkedList

Operations:

push(3)
push(5)
push(8)



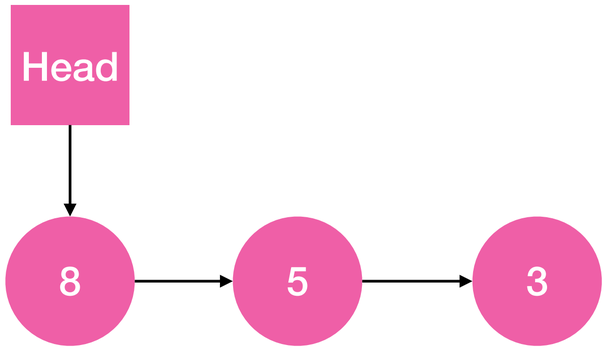
Solution 1

Ok for push O(1) ✅, pop impossible ❌

1.2.2 Stack with a simply LinkedList

Operations:

push(3)
push(5)
push(8)



Solution 2

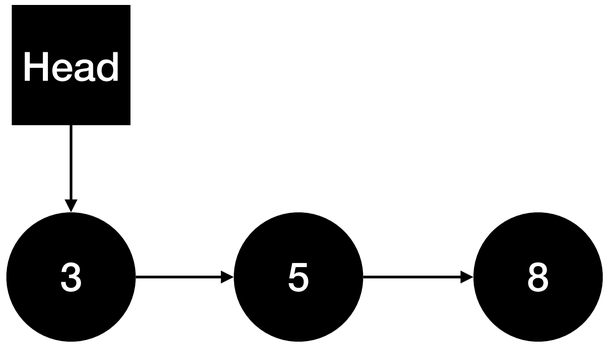
Ok for O(1) ✅, ok for pop O(1) ✅

1.2.2 Stack with a simply LinkedList

Operations

push(3)
push(5)
push(8)

Ok for push O(n) ✅, ok for pop O(n) ✅



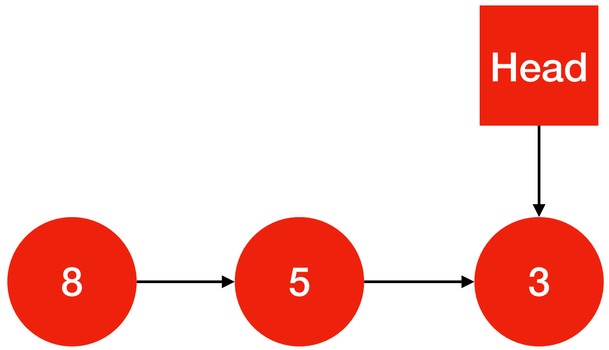
Solution 3

1.2.2 Stack with a simply LinkedList

Operations

push(3)
push(5)
push(8)

push impossible ✗, pop impossible ✗



Solution 4

1.2.3 Evaluation of a post-fix expression

- Example of a post fix expo: “2 3 1 * + 9 *” equivalent to “((3*1)+2)*9”

- These expressions can be easily evaluated using a:
  - Queue (FIFO) ?
  - Stack (LIFO) ?
  - A tree ?

Post-fix evaluation: Algorithm

```java
public static int evaluate(char[] input) {
    Stack<Integer> stack = new Stack<>();
    int i = 0;

    while (i < input.length) {
        char current = input[i];

        // If the current character is a number
        if (Character.isDigit(current)) {
            // Convert char to int and push onto stack
            stack.push(Character.getNumericValue(current));
        }
        // If the current character is an operator
        else if (isOperator(current)) {
            // Pop two numbers from the stack
            int operand2 = stack.pop();  // Second operand
            int operand1 = stack.pop();  // First operand

            // Apply the operator and push the result back onto the stack
            int result = applyOperator(operand1, operand2, current);
            stack.push(result);
        }
        i++;  // Move to the next character
    }

    // The final result is the last value in the stack
    return stack.pop();
}
```

4	2	+	3	5	1	*	*	-

```java
// Helper method to check if a character is an operator
private static boolean isOperator(char ch) {
    return ch == '+' || ch == '-' || ch == '*' || ch == '/';
}

// Helper method to apply an operator to two operands
private static int applyOperator(int operand1, int operand2, char operator) {
    switch (operator) {
        case '+': return operand1 + operand2;
        case '-': return operand1 - operand2;
        case '*': return operand1 * operand2;
        case '/': return operand1 / operand2;
        default: throw new IllegalArgumentException("Invalid operator: " + operator);
    }
}
```

Time complexity of post-fix evaluation ?

- Assuming push/pop operations in O(1) and *n* the length of the expression:
  - $\Theta(n)$
  - $\Theta(n^2)$
  - $O(n)$
  - $O(n^2)$

1.2.4 Functional List (FList)

- Is **an immutable list** (can never be modified after its creation) recursivce

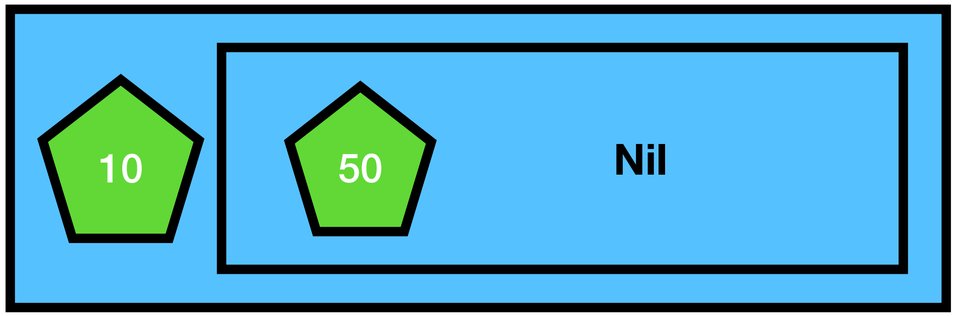
- We can only create it with *cons* method and we can then iterate on it. Example:

```
public static void main(String[] args) {
    FList<Integer> list = FList.nil();

    for (int i = 0; i < 10; i++) {
        list = list.cons(i);
    }

    list = list.map(i -> i+1);
    // will print 10,9,...,1
    for (Integer i: list) {
        System.out.println(i);
    }

    list = list.filter(i -> i%2 == 0);
    // will print 10,...,6,4,2
    for (Integer i: list) {
        System.out.println(i);
    }
}
```

Functional lists: FList

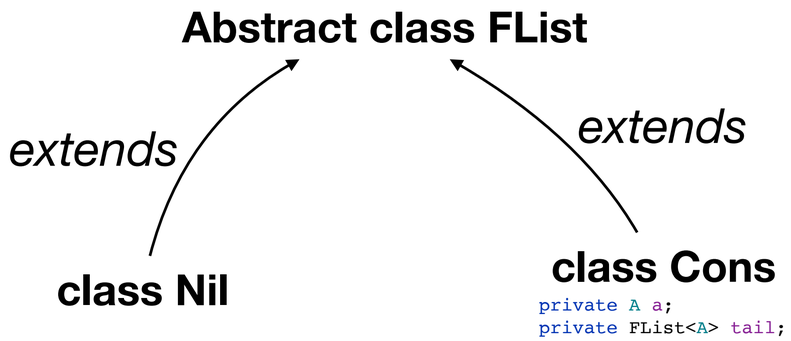
- Recursive definition. A FList is either:
  - An empty list (Nil), or
  - An element followed by an FList

- Example for the list [10,50]



- Our implementation will follow this definition.

Class Diagram



Complexité ?

```
public abstract class FList<A> implements Iterable<A> {
    // creates an empty list

    public static <A> FList<A> nil();

    // prepend a to the list and return the new list
    public final FList<A> cons(final A a);

    public final boolean isNotEmpty();
    public final boolean isEmpty();
    public final int length();
    // return the head element of the list
    public abstract A head();
    // return the tail of the list
    public abstract FList<A> tail();
    // return a list on which each element has been applied function f
    public final <B> FList<B> map(Function<A,B> f);
    // return a list on which only the elements that satisfies predicate are kept
    public final FList<A> filter(Predicate<A> f);
    // return an iterator on the element of the list
    public Iterator<A> iterator();

}
```

Empty list

Add an element (prepend) to the list and return the new list.

Time Complexity ? :

- O(1)
- O(n)
- Θ(n)

What is the space complexity of this method?

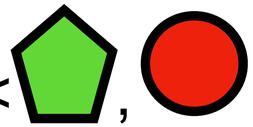
- $\Theta(n)$
- $\Theta(n^2)$
- $O(n)$
- $O(n^2)$

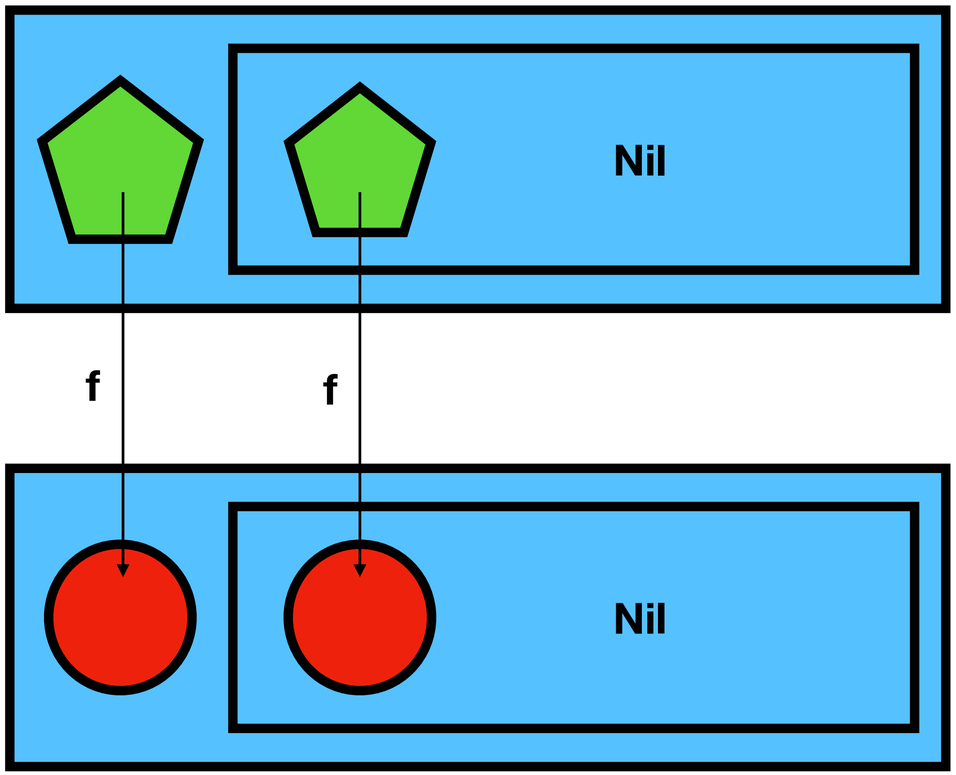
```java
/**
 * @param n the size of the list
 * @return The list [0,1,...,n-1]
 */
public static FList<Integer> rangeList(int n) {
    FList<Integer> list = FList.nil();
    for (int i = n-1; i >= 0; i--) {
        list = list.cons(i);
    }
    return list;
}
```

The « map » method

- When applied to an `FList<A>`, it takes a function `f: A → B` as an argument and reconstructs a list of type `FList<B>`

```
public final <B> FList<B> map(Function<A,B> f)
```

- Example for f  , >



The « map » method

- Example:

```
FList<Integer> list = FList.nil();

for (int i = 9; i >= 0; i--) {
    list = list.cons(i);
}

Function<Integer,String> f = new Function<Integer, String>() {
    @Override
    public String apply(Integer k) {
        String res = "";
        for (int i = 0; i < k; i++) {
            res += "*";
        }
        return res;
    }
};

FList<String> rList = list.map(f);
for (String r: rList) {
    System.out.println(r);
}
```

- TimeComplexity ?
  - $\Theta(n)$
  - $\Theta(n^2)$
  - $O(n)$
  - $O(n^2)$

Syntactical sugar (since Java8) for functions

```java
@FunctionalInterface
public interface Function<T, R> {
    R apply(T t);
}
```

```java
FList<Integer> list = FList.nil();

for (int i = 9; i >= 0; i--) {
    list = list.cons(i);
}

FList<String> rList = list.map(k -> {
    String res = "";
    for (int i = 0; i < k; i++) {
        res += "*";
    }
    return res;
});

for (String r: rList) {
    System.out.println(r);
}
```

=

```java
FList<String> rList = list.map(
  new Function<Integer, String>() {
    @Override
    public String apply(Integer k) {
        String res = "";
        for (int i = 0; i < k; i++) {
            res += "*";
        }
        return res;
    }
});
```

Implicit implementation of the single method of a 'functional' interface

LINFO 1121
DATA STRUCTURES AND ALGORITHMS

UCLouvain

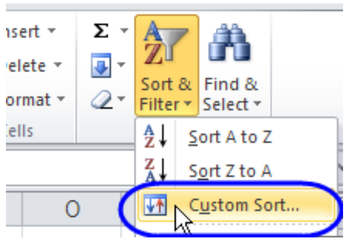
Intro 2

Sorting algorithms and binary search

Pierre Schaus

Sorting data

- One of the most common operations in computer science:



- At the dawn of computing (not so long ago), it was said that 30% of computing resources were spent on sorting.

- Today, this is likely less due to highly efficient sorting algorithms.

Why study sorting algorithms?

- From a theoretical standpoint, sorting algorithms are very interesting and serve as an excellent example for comparing algorithms (memory usage, complexity, etc.).
- They form the foundation of many other algorithms.
- Mastering sorting algorithms is essential as they represent a fundamental algorithmic building block.
- The ideas behind sorting algorithms are quite generic and can be reused to solve other problems (divide and conquer, merging, pivoting, etc.).

Les questions à se poser en lisant

- Why are there so many sorting algorithms?
- Are there advantages and disadvantages to each of them?
- Is it possible to implement a generic sorting algorithm that can sort any object?
- Does the complexity depend on the objects being sorted?
- Can I sort any linear data structure (linked list, array, etc.), or only arrays?
- What is the minimum API a linear data structure must provide to be sortable?
- Can I sort without using additional space?

Les questions à se poser en lisant

- Does O(n log n) vs. O(n^2) make a big difference?
- Are we talking about average-case, worst-case (O, Θ, ~?), or expected complexity?
- Can we ever hope to sort faster than O(n log n)?
  - What exactly does this complexity represent when discussing sorting algorithms?

Les questions à se poser en lisant

- What is meant by a *stable sort*? Why is it (sometimes) important?

- Is it more expensive to perform a lexicographical sort compared to sorting integers?

Tris sur les String

- Strings are very common, and sorting them is also a frequent task.

- Do the algorithms that work best for sorting integers also work best for sorting strings? Why or why not?

- An integer is a string of bits (0s and 1s), so why don’t we sort integers the same way we sort strings?

Important de se procure une copie du livre

Your feeling after this first module?

- Book?
- Course organization?
- Inginious?
- Workload?